



KONTENT ROOM

EIN 83-1404605

**PROYECTO TAGSCREEN:
Test Mock API Back-End
Demo Fútbol**

Diego Alejandro González Vargas
CC: 1.000.065.767

10 de octubre de 2025

Instructivo para Ingeniero Junior - Sistema de Análisis de Fútbol

Objetivo General

Familiarizarse con el sistema de análisis de eventos deportivos y desarrollar una Mock API que simule eventos del partido en tiempo real, específicamente para validar detecciones del sistema ML contra eventos reales del segundo del video que estamos usando como demo.

Contexto del Proyecto

Este proyecto analiza frames del partido Argentina vs Francia (Qatar 2022 Final) usando Machine Learning para detectar:

1. **Reconocimiento facial:** Identifica jugadores (Messi, Mbappé, Di María, etc.)
2. **Clasificación de eventos:** Determina si en la imagen hay un gol o no hay gol
3. **Detección de camisetas:** Identifica equipos (Argentina vs Francia)
4. **Validación temporal:** Verifica si un evento detectado corresponde con eventos reales del partido

El sistema actual tiene endpoints funcionales para análisis ML, pero **falta implementar la Mock API** que simule los eventos reales del partido.

Setup Inicial

Requisitos

- Python 3.9 o superior
- Acceso al repositorio del proyecto
- Video/frame de demo proporcionado

Instalación

```
bash
```

```
# Clonar repositorio (si aplica)
```

```
cd /ruta/del/proyecto
```

Crear ambiente virtual

```
python -m venv venv
```

Activar ambiente

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Instalar dependencias

```
pip install -r requirements.txt
```

Verificar Instalación

```
bash
```

Iniciar servidor

```
python -m app.main
```

En otra terminal, verificar:

```
curl http://localhost:8003/health
```

Respuesta esperada:

```
# {"status": "ok", "models": {...}}
```

Estructura del Proyecto

app/

- └─ api/
 - | └─ routers/
 - | | └─ analyze.py # Endpoint de análisis sin validación
 - | | └─ validate.py # Endpoints de validación (implementados)
 - | | └─ health.py # Health check
 - | | └─ ask.py # Comentarista IA
 - | └─ deps.py # Inyección de dependencias
 - └─ core/
 - | └─ config.py # Configuración
 - └─ ml/
 - | └─ faces/
 - | | └─ recognizer.py # Reconocimiento facial
 - | └─ classifiers/
 - | | └─ goal_clip_keras.py # Clasificador de eventos
 - | └─ detectors/
 - | └─ jerseys.py # Detector de camisetas
 - └─ services/
 - | └─ analysis_service.py # Servicio principal de análisis
 - | └─ match_events_service.py # Servicio de eventos (parcialmente implementado)
 - └─ schemas/
 - | └─ io.py # Modelos de datos (Pydantic)
 - └─ main.py # Punto de entrada

TAREA 1: Estudiar y Documentar el Código

Tiempo estimado: 3-4 horas

Objetivo

Comprender el funcionamiento completo del sistema de reconocimiento de eventos y documentar el flujo de procesamiento.

Parte 1.1: Flujo de Request (1 hora)

Actividades:

1. Analizar el endpoint `/analyze` en `app/api/routers/analyze.py`
2. Seguir el flujo hacia `AnalysisService` en `app/services/analysis_service.py`
3. Identificar los componentes ML que se invocan
4. Documentar el flujo completo

Preguntas a responder:

- ¿Qué sucede cuando un cliente envía una imagen a `/analyze`?
- ¿Qué clases se instancian y en qué orden se ejecutan?
- ¿Qué información devuelve el endpoint?
- ¿Cuál es la estructura del objeto de respuesta?

Entregable: Documento describiendo el flujo paso a paso

Parte 1.2: Clasificación de Eventos (1 hora)

Actividades:

1. Estudiar `app/ml/classifiers/goal_clip_keras.py`
2. Analizar el método `predict()`
3. Entender el rol de CLIP embeddings
4. Identificar las clases de salida

Preguntas a responder:

- ¿Cómo funciona el clasificador de goles?
 - ¿Qué es un "embedding" y para qué se usa?
 - ¿Qué valores puede tener event_class?
 - ¿Qué representa el valor confidence?
 - ¿Cómo se determina el "top_event"?
-

TAREA 2: Implementar Mock API

Objetivo

Diseñar e implementar una Mock API que simule eventos del partido en tiempo real para validar las detecciones del sistema ML, específicamente para el frame del video demo proporcionado.

Contexto Técnico

El sistema actual incluye:

- **Endpoints de validación implementados** (/validate/*)
- **Estructura base en** match_events_service.py
- **Esquemas de datos definidos** en io.py

Lo que falta:

- **Implementación de la Mock API** que devuelva eventos simulados
- **Timeline de eventos** del partido
- **Lógica para simular diferentes escenarios** (gol en vivo, repetición, etc.)

Libertad de Implementación

Tienes completa libertad para:

- Diseñar la estructura de datos como prefieras
- Modificar o crear nuevos endpoints
- Implementar la lógica de simulación que consideres apropiada
- Usar archivos JSON, base de datos en memoria, o cualquier solución
- Proponer mejoras a la arquitectura existente

Importante: No es obligatorio usar los endpoints sugeridos. Puedes proponer y justificar tu propio diseño.